

HBOS • 鸿蒙裸机内核操作系统

HBOS 应用开发手册

HAX Application Development Guide • [.hax / C 版](#)

XINGJI • HBOS OPERATING SYSTEM

2026/06/29 • VERSION 1.0 • HBOS v0.1-beta3

目录 CONTENTS

第 1 章 — 关于、编译与专有类型

1.1 关于本手册

1.2 什么是 HAX 应用 (.hax)

1.3 自动导入机制：把应用放进 ./app

1.4 编译为 HAX 应用

1.5 专有类型概览

1.6 应用元数据 HAX_APP()

第 2 章 — HAX SDK (终端/通用)

2.1 文本输入输出

2.2 文件操作

2.3 系统服务

第 3 章 — 运行与发现

3.1 TUI 与 GUI 应用

3.2 apps / run 命令

3.3 在图形桌面运行

第 4 章 — 完整示例

4.1 hello — 最小应用

4.2 guess — 猜数字游戏

附录 A — 底层 POSIX 系统调用

第 1 章 ABOUT, COMPILE & TYPES

本章介绍 HAX 应用格式、它的自动导入机制、如何编译，以及 SDK 提供的专有类型。

1.1 关于本手册

本手册面向想在 HBOS 上开发应用的用户。HBOS 提供一套名为 **HAX** (HBOS Application eXecutable) 的应用机制：你只需用 C 写一个 `main()`，包含 `<hax.h>`，把源码放进项目的 `./app` 目录，执行 `make`，应用就会被编译、打包并在系统启动后 **自动出现**——无需改动内核、无需手动注册。应用可在终端 (TUI) 运行，也可从图形桌面 (GUI) 启动。

1.2 什么是 HAX 应用 (.hax)

一个 `.hax` 文件在底层就是**标准 ELF64 用户态可执行程序**，运行在 `0x1000000000` 起始的用户地址空间，通过 `int 0x80` 进入内核。它与普通 ELF 的唯一区别是携带了一段自描述元数据 `.haxmeta`，使系统能识别应用名、描述和类型。扩展名 `.hax` 是 HBOS 专用的标记，与示范规范中的 `.epf` / `.elf` 不同，用于让构建系统区分“需要自动导入的应用”。

1.3 自动导入机制：把应用放进 ./app

核心逻辑非常简单：只要 `./app` 目录里有编译产物 (`.hax`)，它就会被加入系统。

步骤	发生了什么
① 编译	<code>make</code> 把每个 <code>app/*.c</code> 用 HAX 运行时链接为 <code>build/app/<名>.hax</code>
② 打包	<code>tools/genhax.py</code> 读取每个 <code>.hax</code> 的 <code>.haxmeta</code> 段，拼接二进制并生成应用清单
③ 嵌入	清单与二进制 blob 一并编译进内核镜像
④ 注册	系统启动后，应用表立即可用——apps 能看到它， <code>run <名></code> 能运行它

你也可以把别处**预编译好的** .hax（标准 ELF64 + .haxmeta）直接丢进 ./app，下次 make 会一并打包进系统。

1.4 编译为 HAX 应用

通常你什么都不用做——make 会自动构建 ./app 下的所有应用。若要单独构建：

```
make hax-apps      # 仅构建所有 .hax 应用

make               # 构建整个系统（含 .hax 自动打包）
```

手动编译时，HAX 应用等价于一个链接了 HBOS 用户态 libc + crt0 的 ELF64：

```
gcc -c -m64 -ffreestanding -fno-stack-protector -fno-pie -mno-red-zone \
    -mno-sse -mno-sse2 -Iapp/include -Isrc/user -Isrc/user/libc \
    app/hello.c -o hello.o

ld -m elf_x86_64 -static -nostdlib -T user.ld \
    <用户态 libc 目标文件...> hello.o -o hello.hax
```

1.5 专有类型概览

<hax.h> 定义了一组统一的定宽类型与颜色类型：

类型	长度（字节）	说明
HI8 / HU8	1	8 位有/无符号整型
HI16 / HU16	2	16 位有/无符号整型
HI32 / HU32	4	32 位有/无符号整型
HI64 / HU64	8	64 位有/无符号整型

HCOLOR	4	0xRRGGBB 颜色值
hax_meta_t	104	应用自描述元数据（见 1.6）

1.6 应用元数据 HAX_APP()

在源文件顶层调用一次 HAX_APP() 宏，声明应用名、描述与类型。它会把一个 hax_meta_t 结构放进 .haxmeta 段，仅供构建工具识别，不参与运行时加载。

```
HAX_APP("hello", "我的第一个 HBOS 应用", HAX_KIND_TUI);
```

参数	说明
name	应用名（≤31 字节），run 命令使用，须唯一
desc	一句话描述（≤63 字节）
kind	HAX_KIND_TUI / HAX_KIND_GUI / HAX_KIND_BOTH

第 2 章 HAX SDK

SDK 是对 HBOS 用户态 libc 与系统调用的轻量封装，让常见操作一行搞定。所有函数声明在 `<hax.h>`。

2.1 文本输入输出

`void hax_print(const char *s);`

输出字符串，不换行。

`void hax_println(const char *s);`

输出字符串并换行。

`hax_printf(fmt, ...);`

格式化输出，用法同标准 `printf`。支持 `%d %u %x %s %c %p` 等。

```
hax_printf("PID=%d 名称=%s\n", hax_pid(), "demo");
```

`int hax_input(char *buf, int cap);`

从标准输入读取一行到 `buf`（自动去除结尾换行），返回字符串长度；读到 EOF 返回 -1。

```
char name[64];

hax_print("你叫什么名字? ");

if (hax_input(name, sizeof(name)) >= 0)

    hax_printf("你好, %s! \n", name);
```

`int hax_getch(void);`

读取一个字符，无字符返回 -1。

2.2 文件操作

long hax_read_file(const char *path, void *buf, long cap);

读取整个文件到 buf（最多 cap 字节），返回读到的字节数；失败返回 -1。

long hax_write_file(const char *path, const void *buf, long len);

把 buf 的 len 字节写入文件（覆盖创建），返回写入字节数；失败返回 -1。

```
const char *msg = "hello hbos\n";

hax_write_file("/tmp/note.txt", msg, 11);


char buf[256];

long n = hax_read_file("/tmp/note.txt", buf, sizeof(buf));

hax_printf("读到 %ld 字节\n", n);
```

2.3 系统服务

函数	说明
void hax_sleep(unsigned sec);	休眠指定秒数
void hax_exit(int code);	以状态码退出应用
int hax_pid(void);	返回当前进程 ID

更底层的能力（fork/socket/mmap 等）可直接使用 libc 或 POSIX 系统调用，见附录 A。

第 3 章 RUN & DISCOVERY

3.1 TUI 与 GUI 应用

kind	含义	入口
HAX_KIND_TUI	终端应用	命令行 run，或在 GUI 终端窗口运行
HAX_KIND_GUI	图形/桌面应用	出现在桌面启动器，启动到终端窗口运行
HAX_KIND_BOTH	两者皆可	两种入口都注册

当前版本中，GUI 类应用在 HBOS 图形终端窗口内运行其文本界面。直接对窗口 framebuffer 绘制的 GUI

窗体 API（创建窗口、画点/线/文本、按钮控件等）为后续版本预留。

3.2 apps / run 命令

```
HBOS> apps

hello - 最小 HAX 示例：问候并回显参数 [TUI .hax]

guess - 猜数字小游戏（1-100）      [GUI .hax]


HBOS> run hello 星际 你好

你好，HBOS！ 这是一个 .hax 应用。

我的 PID 是 7

收到参数: 星际 你好
```

命令	作用
apps	列出所有应用（含自动发现的 .hax，带 [TUI]/[GUI] 标签）

`run <名> [参数...]` 运行应用；先匹配内建应用，未命中再匹配 .hax 应用

3.3 在图形桌面运行

在 GUI 桌面打开“终端”窗口，即可像命令行一样输入 `apps` 与 `run <名>`。GUI 类应用的输出会显示在终端窗口中。

第 4 章 EXAMPLES

4.1 hello —— 最小应用

```
#include <hax.h>

HAX_APP("hello", "最小 HAX 示例：问候并回显参数", HAX_KIND_TUI);

int main(int argc, char **argv) {

    hax_println("你好，HBOS！ 这是一个 .hax 应用。");

    hax_printf("我的 PID 是 %d\n", hax_pid());

    if (argc > 1) {

        hax_print("收到参数:");

        for (int i = 1; i < argc; i++) hax_printf(" %s", argv[i]);

        hax_print("\n");

    }

    return 0;

}
```

4.2 guess —— 猜数字游戏

```
#include <hax.h>

HAX_APP("guess", "猜数字小游戏（1-100）", HAX_KIND_GUI);
```

```
int main(int argc, char **argv) {

    (void)argc; (void)argv;

    int secret = (hax_pid() * 37 + 11) % 100 + 1; /* 简易种子 */

    char line[32];

    hax_println("我想了一个 1-100 的数，来猜！(q 退出)");

    for (int tries = 1; ; tries++) {

        hax_print("你的猜测> ");

        if (hax_input(line, sizeof(line)) < 0 || line[0]=='q') break;

        int v = atoi(line);

        if (v < secret) hax_println("太小了。");

        else if (v > secret) hax_println("太大了。");

        else { hax_printf("猜对了！用了 %d 次。\\n", tries); break; }

    }

    return 0;

}
```

提示：把这两个文件放进 ./app，执行 make，重启系统后 apps 即可看到它们。

附录 A POSIX SYSCALLS

除 SDK 外，HAX 应用可直接使用 HBOS 用户态 libc 暴露的 POSIX 接口（声明在 <libc/syscall.h> 与 <libc/stdio.h> 等）。下表为常用子集。

类别	接口
文件 I/O	open close read write lseek fstat stat unlink
进程控制	getpid getppid exit sleep usleep fork execve waitpid kill
文件系统	mkdir rmdir getcwd chdir access ftruncate readlink
内存	sbrk brk mmap munmap mprotect malloc free
网络	socket bind listen accept connect send recv
标准库	printf fopen fread fwrite fgets atoi memcpy strlen ...

网络相关：HBOS 在链路未提供 DNS 时会自动回退公共 DNS（8.8.8.8），因此即便“网络未配置

DNS”，域名解析仍可工作。